Misión: el robot de CI del modelo de riesgo (Sesión 4)

El caso. El banco ya publicó la **v1.0** del modelo de riesgo de crédito. Pero en la sesión pasada un error de cálculo (AUC 0.27) llegó a producción y solo lo detectó el área de Riesgos. El comité pidió algo simple: **cada cambio debe reconstruir el modelo desde cero, en una máquina limpia, de forma automática**. Hoy construyen ese robot con GitHub Actions.

Cómo trabajamos (parejas · 45 min)

Rol	Qué hace
Driver	Comparte pantalla y escribe el YAML.
Navigator	Lee esta guía, revisa la indentación, busca en la documentación y lee los logs.

Cambien de rol en cada misión. Las Misiones 1 a 4 se hacen en **un solo repo** (el del primer Driver); quien sea Driver en cada misión comparte pantalla y edita ese repo. Antes de la Misión 5, el otro integrante copia el workflow terminado a **su propio repo** (1 minuto).

Antes de empezar (cada uno, una sola vez, aunque solo usen un repo hasta la Misión 4):

1. En la página del repo plantilla en GitHub: **Use this template** → **Create a new repository**.
   - Owner: la organización del curso · Nombre: `actions-<tu-usuario>` · Visibilidad: **Public**.
2. **Settings** → **Collaborators and teams**: agrega a tu pareja con permiso **Write** (lo necesitarán en la Misión 5).
3. Puedes escribir el workflow **directamente en GitHub** (*Add file* → *Create new file*) o en PyCharm y luego `git push`. Ambas formas valen.

> 💡 Los workflows **solo corren en GitHub**, nunca en tu laptop. Si lo editas en local, tienes que hacer `push` para verlo correr.

Misión 1 · El robot despierta · ~10 min

Crea el archivo `.github/workflows/model-ci-cd.yml` (respeta la ruta exacta: carpeta `.github`, con punto, y subcarpeta `workflows`).

```yaml
name: Model CI/CD
on: workflow_dispatch
jobs:
  model-ci:
    runs-on: ubuntu-latest
```

```
    steps:
      - name: Traer el código
        uses: actions/checkout@v6
      - name: Instalar Python
        uses: actions/setup-python@v6
        with:
          python-version: '3.12'
      - name: Instalar dependencias
        run: pip install -r requirements.txt
```

1. Commit en `main`.
2. Pestaña **Actions** → workflow **Model CI/CD** → **Run workflow**.
3. Abre la ejecución y despliega cada step para ver su log.

✅ **Listo cuando:** ves el job `model-ci` en verde y en el log de *Instalar dependencias* aparece `Successfully installed ... pandas ... scikit-learn`.

> ❓ **Pregunta para la pareja:** ¿por qué escribimos `'3.12'` entre comillas? *(Pista: ¿cómo leería YAML el número `3.10`?)*

Misión 2 · El pipeline completo en una máquina limpia · ~10 min

Agrega al final de `steps` un step por cada script del pipeline, **en orden**:

```
      - name: Descargar datos
        run: python src/0-download-data.py
      - name: Separar train y test
        run: python src/1-split-dataset.py
      - name: Preprocesar train
        run: python src/2-preprocess-dataset-train.py
      - name: Preparar test
        run: python src/3-prepare-dataset-test.py
      - name: Entrenar y evaluar modelos
        run: python src/4-train-evaluate-models.py
      - name: Seleccionar mejor modelo
        run: python src/5-select-best-model.py
      - name: Mostrar métricas
        run: cat outputs/train/metrics/train_test_metrics.csv
```

Ejecútalo de nuevo con **Run workflow** (tarda unos minutos: descarga los datos y entrena).

✅ **Listo cuando:** el step *Mostrar métricas* imprime el AUC de train y test. Anótenlo en el chat.

> ❓ ¿Por qué el robot tiene que **descargar los datos** cada vez, si tú ya los tienes en tu laptop?

Misión 3 · De CI a CD: dos jobs en secuencia · ~8 min

1. Agrega un segundo job **al mismo nivel que** `model-ci` (misma indentación):

```yaml
  model-cd:
    runs-on: ubuntu-latest
    steps:
      - name: Desplegar modelo
        run: echo "Desplegando el modelo de riesgo en AWS... (simulado)"
```

2. Ejecuta. **Antes de mirar, predigan:** ¿en qué orden corren los dos jobs?
3. Ahora agrega `needs: model-ci` dentro de `model-cd` (antes de `runs-on`) y vuelve a ejecutar.

✅ **Listo cuando:** el grafo de la ejecución muestra `model-ci → model-cd` unidos por una flecha.

Misión 4 · Automático · ~5 min

1. Cambia el disparador para que corra **con cada push** y conserve el botón manual:

```yaml
on: [push, workflow_dispatch]
```

2. Haz un cambio pequeño cualquiera (por ejemplo, en el `README.md`) y haz commit.
3. Ve a **Actions**: el workflow arrancó solo. El nombre de la ejecución es tu mensaje de commit.

✅ **Listo cuando:** ves una ejecución disparada por `push` (no por `workflow_dispatch`).

> ❓ Acabas de re-entrenar el modelo por cambiar una línea del README. ¿Tiene sentido? *(Lo resolvemos en la Sesión 5.)*

Misión 5 · 🚨 Sala de emergencias · ~10 min

Primero, el integrante que aún no tiene el workflow lo copia a su repo `actions-<usuario>` (mismo archivo, misma ruta) y lo ejecuta una vez para confirmar que está en verde.

Ahora cada uno **sabotea el workflow de su pareja** (sin decirle qué hizo). Elige **uno** de estos sabotajes, haz commit directamente en el repo de tu pareja y avísale: *"rompí algo, encuéntralo"*.

#	Sabotaje
1	Borra el step `Traer el código` (checkout).
2	Borra el step `Instalar dependencias` .

#	Sabotaje
3	Cambia el nombre de un script, por ejemplo `src/4-train-model.py` .
4	Quita 2 espacios de indentación a la línea `steps:` de `model-ci` .

El diagnosticador responde en el chat, **solo mirando la pestaña Actions**:

1. ¿Qué **job** falló? ¿Qué **step**?
2. ¿Qué dice la **última línea roja** del log?
3. ¿Cuál es la causa y cómo lo arreglas? Arréglalo y vuelve a ejecutar.

✅ **Listo cuando:** ambos repos vuelven a estar en verde.

Retos extra

- Agrega un tercer job `notificar` que corra **después de los otros dos** e imprima `¡Pipeline completo!` .
- Agrega un disparador `schedule` para re-entrenar cada lunes a las 6:00 a.m. de Lima (*pista: el cron usa hora UTC*).
- Agrega `cache: pip` dentro del `with:` de `setup-python` y compara el tiempo del step *Instalar dependencias* entre dos ejecuciones.
- Explora el Marketplace: encuentra una action que te sería útil en tu trabajo.

Para el docente: qué verá cada sabotaje

#	Dónde falla	Última línea roja (aprox.)
1	`model-ci` → *Instalar dependencias*	`ERROR: Could not open requirements file: [Errno 2] No such file or directory: 'requirements.txt'`
2	`model-ci` → *Separar train y test* (la descarga sí pasa: solo usa librerías estándar)	`ModuleNotFoundError: No module named 'pandas'`
3	`model-ci` → *Entrenar y evaluar modelos*	`python: can't open file '.../src/4-train-model.py': [Errno 2] No such file or directory`
4	El workflow ni siquiera arranca	Mensaje *Invalid workflow file* con la línea del error de YAML